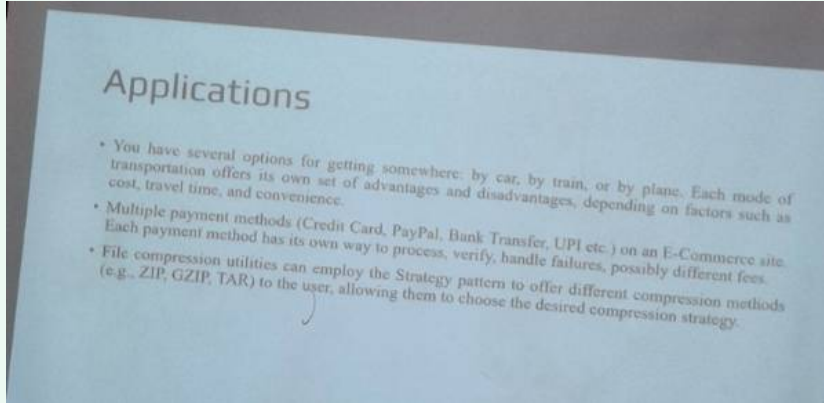


Strategy pattern in Java



Implement the first task with strategy pattern in java

Below is a **simple, clear Java implementation of the first application using the Strategy Pattern.**

Task:

“You have several options for getting somewhere: by car, train, or plane. Each mode has different cost, speed, and convenience.”

Goal:

Use **Strategy Pattern** so that a traveler can choose different transportation strategies at runtime.

Strategy Pattern Example — Transportation Modes

1. Strategy Interface

```
java

// Strategy interface
public interface TransportationStrategy {
    void travel(String source, String destination);
}
```

2. Concrete Strategies

Car

```
java

public class CarStrategy implements TransportationStrategy {
    @Override
    public void travel(String source, String destination) {
        System.out.println("Traveling from " + source + " to " + destination +
" by Car.");
        System.out.println("Cost: Medium | Time: Medium | Convenience: High");
    }
}
```

Train

```
java

public class TrainStrategy implements TransportationStrategy {
    @Override
    public void travel(String source, String destination) {
        System.out.println("Traveling from " + source + " to " + destination +
" by Train.");
        System.out.println("Cost: Low | Time: Medium | Convenience: Medium");
    }
}
```

Plane

```
java

public class PlaneStrategy implements TransportationStrategy {
    @Override
    public void travel(String source, String destination) {
        System.out.println("Traveling from " + source + " to " + destination +
" by Plane.");
        System.out.println("Cost: High | Time: Fast | Convenience: Medium");
    }
}
```

3. Context Class

```
java

public class TravelContext {
    private TransportationStrategy strategy;

    // allows switching strategy at runtime
    public void setStrategy(TransportationStrategy strategy) {
        this.strategy = strategy;
    }

    public void travel(String from, String to) {
        if (strategy == null) {
            System.out.println("No travel strategy selected.");
            return;
        }
        strategy.travel(from, to);
    }
}
```

4. Main Class (Test Program)

```
java

public class Main {
    public static void main(String[] args) {

        TravelContext context = new TravelContext();

        // Choose Car
        context.setStrategy(new CarStrategy());
        context.travel("Dhaka", "Chittagong");

        System.out.println();

        // Choose Train
        context.setStrategy(new TrainStrategy());
        context.travel("Dhaka", "Sylhet");

        System.out.println();

        // Choose Plane
        context.setStrategy(new PlaneStrategy());
        context.travel("Dhaka", "Cox's Bazar");
    }
}
```

 **Output Will Look Like**

vbnet

Traveling **from** Dhaka **to** Chittagong **by** Car.
Cost: Medium | **Time:** Medium | **Convenience:** High

Traveling **from** Dhaka **to** Sylhet **by** Train.
Cost: Low | **Time:** Medium | **Convenience:** Medium

Traveling **from** Dhaka **to** Cox's Bazar **by** Plane.
Cost: High | **Time:** Fast | **Convenience:** Medium

If you want, I can also:

- ✓ Add user input (Scanner)
- ✓ Add advanced attributes (time, price, comfort)
- ✓ Add UML diagram
- ✓ Convert to FastAPI-like structure in Java (Spring Boot)

Just tell me!



give another full example of strategy pattern

Here is **another full, clean, understandable Strategy Pattern example** — this time using **different discount strategies in a shopping cart**.

It is simple, realistic, and commonly used in software design interviews.

Strategy Pattern Example: Discount Strategies in Shopping

Scenario

A shopping cart can apply different discount strategies:

- **No Discount**
- **Percentage Discount**
- **Fixed Amount Discount**

- **Seasonal Discount**

The user can choose the discount at runtime.

✓ 1. Strategy Interface

```
java

public interface DiscountStrategy {
    double applyDiscount(double price);
}
```

✓ 2. Concrete Strategies

No Discount

```
java

public class NoDiscountStrategy implements DiscountStrategy {
    @Override
    public double applyDiscount(double price) {
        return price;
    }
}
```

Percentage Discount

```
java

public class PercentageDiscountStrategy implements DiscountStrategy {
    private double percent;

    public PercentageDiscountStrategy(double percent) {
        this.percent = percent;
    }

    @Override
    public double applyDiscount(double price) {
        return price - (price * (percent / 100));
    }
}
```

Fixed Discount

```
java

public class FixedDiscountStrategy implements DiscountStrategy {
    private double amount;

    public FixedDiscountStrategy(double amount) {
        this.amount = amount;
    }

    @Override
    public double applyDiscount(double price) {
        double finalPrice = price - amount;
        return finalPrice > 0 ? finalPrice : 0;
    }
}
```

Seasonal Discount

```
java

public class SeasonalDiscountStrategy implements DiscountStrategy {
    @Override
    public double applyDiscount(double price) {
        return price * 0.80; // 20% off
    }
}
```

3. Context Class

```
java

public class ShoppingCart {
    private DiscountStrategy discountStrategy;

    public void setDiscountStrategy(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    public void checkout(double price) {
        if (discountStrategy == null) {
            System.out.println("No discount selected.");
            return;
        }

        double finalPrice = discountStrategy.applyDiscount(price);
        System.out.println("Original Price: " + price);
        System.out.println("Final Price after Discount: " + finalPrice);
    }
}
```

✓ 4. Main Class (Test Program)

java

```
public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        double price = 1000;

        // 1. No Discount
        cart.setDiscountStrategy(new NoDiscountStrategy());
        cart.checkout(price);

        System.out.println();

        // 2. 10% Discount
        cart.setDiscountStrategy(new PercentageDiscountStrategy(10));
        cart.checkout(price);

        System.out.println();

        // 3. Fixed 200 taka discount
        cart.setDiscountStrategy(new FixedDiscountStrategy(200));
        cart.checkout(price);

        System.out.println();

        // 4. Seasonal 20% Discount
        cart.setDiscountStrategy(new SeasonalDiscountStrategy());
        cart.checkout(price);
    }
}
```

Sample Output

yaml

```
Original Price: 1000.0
Final Price after Discount: 1000.0

Original Price: 1000.0
Final Price after Discount: 900.0

Original Price: 1000.0
Final Price after Discount: 800.0

Original Price: 1000.0
Final Price after Discount: 800.0
```

Want more examples?

I can also give:

- ✓ Strategy pattern for **sorting algorithms**
- ✓ Strategy for **payment methods**
- ✓ Strategy for **file compression (ZIP, GZIP)**
- ✓ Strategy pattern diagram (UML)

Just tell me!



make this example easy, keep the classes and methods same but do less printing or user input for better understanding

Sure! Here is the **same discount example**, but made **much simpler**, with **minimal printing, no extra text**, and **cleaner code** so it's easier to understand.

Classes and method names remain the same — only simplified.

✓ 1. Strategy Interface

```
java

public interface DiscountStrategy {
    double applyDiscount(double price);
}
```

✓ 2. Concrete Strategies

No Discount

```
java

public class NoDiscountStrategy implements DiscountStrategy {
    @Override
    public double applyDiscount(double price) {
        return price;
    }
}
```


Percentage Discount

```
java

public class PercentageDiscountStrategy implements DiscountStrategy {
    private double percent;

    public PercentageDiscountStrategy(double percent) {
        this.percent = percent;
    }

    @Override
    public double applyDiscount(double price) {
        return price - (price * percent / 100);
    }
}
```

Fixed Amount Discount

```
java

public class FixedDiscountStrategy implements DiscountStrategy {
    private double amount;

    public FixedDiscountStrategy(double amount) {
        this.amount = amount;
    }

    @Override
    public double applyDiscount(double price) {
        return price - amount;
    }
}
```

Seasonal Discount

```
java

public class SeasonalDiscountStrategy implements DiscountStrategy {
    @Override
    public double applyDiscount(double price) {
        return price * 0.8; // 20% off
    }
}
```

3. Context Class

java

```
public class ShoppingCart {
    private DiscountStrategy discountStrategy;

    public void setDiscountStrategy(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    public double checkout(double price) {
        return discountStrategy.applyDiscount(price);
    }
}
```

✓ 4. Main Class (Very Simple Output)

java

```
public class Main {
    public static void main(String[] args) {

        ShoppingCart cart = new ShoppingCart();
        double price = 1000;

        cart.setDiscountStrategy(new NoDiscountStrategy());
        System.out.println(cart.checkout(price));

        cart.setDiscountStrategy(new PercentageDiscountStrategy(10));
        System.out.println(cart.checkout(price));

        cart.setDiscountStrategy(new FixedDiscountStrategy(200));
        System.out.println(cart.checkout(price));

        cart.setDiscountStrategy(new SeasonalDiscountStrategy());
        System.out.println(cart.checkout(price));
    }
}
```

✓ Output (clean and minimal)

yaml

```
1000.0
900.0
800.0
800.0
```

If you want, I can also make this example:

- ✓ even shorter
- ✓ with UML diagram
- ✓ with only 2 strategies to make it ultra-easy

Just tell me!



explain me strategy pattern in dp

Here is a **very clear and beginner-friendly explanation** of the **Strategy Pattern** in **Design Patterns (DP)**.

What is the Strategy Pattern?

The **Strategy Pattern** is a behavioral design pattern that lets you **choose different algorithms or behaviors at runtime**.

Instead of writing many **if-else** or **switch** statements, you put each behavior in a separate class and switch between them easily.

Simple Idea

Think of using different **tools** for different jobs:

- You use a *knife* to cut vegetables.
- You use *scissors* to cut paper.
- You use a *saw* to cut wood.

The action is the same: "cut something".
But the *strategy* (tool) changes.



Formal Definition

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable.

That means:

- ✓ You can pick any algorithm at runtime.
- ✓ All strategies follow the same interface.
- ✓ You can add new strategies without changing existing code.

When to use Strategy Pattern?

Use it when:

1. You have **multiple ways** of doing the same task.
2. You keep writing **lots of if/else** or **switch** statements.
3. You want to **change behavior at runtime**.
4. You want to **add new behaviors easily**.

Real-life example (Simple)

You want to travel somewhere:

- **Car**
- **Train**
- **Plane**

Same action: **travel()**, but different strategies.

```
java

interface Transport {
    void travel();
}

class Car implements Transport { public void travel() {
System.out.println("Car"); } }
class Train implements Transport { public void travel() {
System.out.println("Train"); } }
class Plane implements Transport { public void travel() {
System.out.println("Plane"); } }

class Traveler {
```

```
Transport strategy;  
void setStrategy(Transport s) { strategy = s; }  
void go() { strategy.travel(); }  
}
```

This avoids:

```
java  
  
if(type == "car") ...  
else if(type == "train") ...  
else if(type == "plane") ...
```

Structure (Easy Explanation)

1. Strategy Interface

Defines what all strategies must do.

2. Concrete Strategies

Different implementations of the interface.

3. Context

The main class that uses the strategies.



Analogy

Think of the **Context** as a juice maker machine and **Strategy** as different fruit flavors.

The machine (context) does not change — you only change the fruit (strategy).



Benefits

- ✓ Removes big **if-else** blocks
- ✓ Easy to change behavior

- ✓ New strategies can be added without touching old code
 - ✓ Makes code cleaner and organized
-

! Why is it used in Design Patterns (DP)?

Because it supports **OOP principles**:

✓ Open-Closed Principle

You can add new strategies **without modifying** existing code.

✓ Single Responsibility Principle

Each strategy class has **one job**.

✓ Dependency Injection

Context depends on **the interface**, not concrete classes.

Summary (in one line)

Strategy Pattern allows a class to **change its behavior by switching different algorithm objects** at runtime.

If you want, I can also:

- ✓ explain with a diagram
- ✓ show ultra-simple Java example
- ✓ compare with state pattern
- ✓ show when NOT to use strategy pattern

Just tell me!



give me some theoretical questions and answers from the topic strategy pattern

Here are **clear, exam-style theoretical questions and answers** on the **Strategy Pattern**.

These are perfect for viva, quizzes, and short theoretical exams.

Strategy Pattern — Theoretical Q&A

1. What is the Strategy Pattern?

Answer:

The Strategy Pattern is a behavioral design pattern that allows selecting an algorithm or behavior at runtime. It encapsulates different algorithms inside separate classes and makes them interchangeable.

2. Why is the Strategy Pattern used?

Answer:

It is used to avoid long **if-else** or **switch** statements, provide flexibility in choosing algorithms, and allow adding new behaviors without modifying existing code. It keeps the code clean and extensible.

3. What are the main components of the Strategy Pattern?

Answer:

- **Strategy Interface** – defines the common method for all strategies.
 - **Concrete Strategies** – actual implementations of the algorithm.
 - **Context Class** – uses a strategy object to perform the operation.
-

4. How does Strategy Pattern support the Open-Closed Principle?

Answer:

Because you can add new strategies (new classes) without modifying existing code. The context works with the interface, so it does not change when new behaviors are added.

5. Give a real-life example of the Strategy Pattern.

Answer:

Transportation choice: car, train, or plane.

All perform the same action “travel”, but with different implementations. The user can switch the travel method at runtime.

6. What problem does the Strategy Pattern solve?

Answer:

It solves the problem of handling multiple algorithms that can be swapped easily without rewriting the logic. It prevents complex conditional statements and makes behavior modular.

7. When should you use the Strategy Pattern?

Answer:

Use it when:

- There are multiple ways to do the same task.
 - You want to change behavior dynamically.
 - You want to avoid nested **if-else** statements.
 - You expect to add more algorithms in the future.
-

8. What is the difference between Strategy Pattern and State Pattern?

Answer:

- **Strategy Pattern:** The algorithm changes based on the user's choice; change is **intentional**.
 - **State Pattern:** Behavior changes automatically based on state; change is **driven by internal state**.
-

9. Can Strategy Pattern cause too many classes?

Answer:

Yes. One drawback of the pattern is that each algorithm requires its own class, which can increase the number of classes in the project.

10. Why must all strategies share the same interface?

Answer:

Because the context class must be able to use any strategy interchangeably. The interface ensures all strategies have the same method signature.

11. What is the role of the Context class?

Answer:

The Context holds a reference to a Strategy and delegates the actual operation to the strategy's method. It does not implement the behavior itself.

12. How does Strategy Pattern improve code maintainability?

Answer:

By separating algorithms into individual classes, making the code easier to modify, test, and extend. Each behavior can be changed without affecting other parts of the system.

13. Is Strategy Pattern compile-time or runtime polymorphism?

Answer:

Runtime polymorphism.

The strategy object can be changed while the program is running.

14. Give a simple example of where Strategy may be used in software systems.

Answer:

Payment systems: credit card, PayPal, bank transfer.

Each payment method is a different strategy.

15. What happens if no strategy is set in the Context?

Answer:

The context cannot perform the operation because the behavior is delegated to the strategy. Usually, a default strategy or a null check is required.

If you want, I can also give:

- ✓ MCQ questions
- ✓ Short notes
- ✓ Diagram-based questions
- ✓ Coding questions on strategy pattern